Common Table Expressions (CTEs) in SQL

Common Table Expressions (CTEs) are a powerful feature in SQL that enhance the readability, maintainability, and organization of complex queries. A CTE defines a temporary, named result set that you can reference within a single SQL statement (SELECT, INSERT, UPDATE, or DELETE). This temporary result set is not stored as part of the database schema; it exists only for the duration of the query execution.

Benefits of Using CTEs

- **Readability:** CTEs break down complex queries into smaller, logical, and more manageable parts, making the SQL code easier to understand.
- **Reusability:** A CTE can be referenced multiple times within the same query, avoiding repetitive code.
- **Recursion:** CTEs are essential for performing recursive queries, such as traversing hierarchical data (e.g., organizational charts or bill of materials).
- **Modularity:** They allow you to define intermediate result sets, which can be useful for debugging and testing.

Syntax of a CTE

The basic syntax for a CTE is as follows:WITH CteName (Column1, Column2, ...) AS (

```
    -- CTE definition query

    SELECT Column1, Column2, ...

    FROM TableName

    WHERE Condition

)

-- Main query that references the CTE

SELECT *
```

FROM CteName

WHERE AnotherCondition;

Let's explore some examples to illustrate the practical application of CTEs.

Examples of CTEs

Example 1: Simplifying Subqueries

Consider a scenario where you want to find employees who earn more than the average salary. Without a CTE, you might use a subquery.

Without CTE:SELECT employee_id, employee_name, salary

FROM employees

WHERE salary > (SELECT AVG(salary) FROM employees);

With CTE:

Using a CTE makes the query more structured and readable:WITH AverageSalary AS (

SELECT AVG(salary) AS avg_emp_salary

FROM employees

)

SELECT e.employee_id, e.employee_name, e.salary

FROM employees AS e, AverageSalary AS avg_s

WHERE e.salary > avg_s.avg_emp_salary;

Example 2: Calculating Running Totals

CTEs are very useful for calculating running totals or cumulative sums.

Scenario: Calculate the running total of order amounts for each customer.WITH CustomerOrderTotals AS (

```
    SELECT
        customer_id,
        order_date,
        order_amount,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS rn
    FROM orders
)
SELECT
    cot.customer_id,
    cot.order_date,
    cot.order_amount,
    SUM(cot2.order_amount) OVER (PARTITION BY cot.customer_id ORDER BY
cot.order_date) AS running_total
FROM CustomerOrderTotals AS cot
JOIN CustomerOrderTotals AS cot2
    ON cot.customer_id = cot2.customer_id AND cot.rn >= cot2.rn
ORDER BY cot.customer_id, cot.order_date;
```

Example 3: Finding Top N Records per Group

CTEs with window functions are excellent for finding the top N records within specific groups.

Scenario: Find the top 3 highest-paid employees in each department.

```
WITH RankedEmployees AS (
    SELECT
```

```
    employee_id,
    employee_name,
    department,
    salary,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_num
  FROM employees
)
SELECT
  employee_id,
  employee_name,
  department,
  salary
FROM RankedEmployees
WHERE rank_num <= 3;
```

Example 4: Recursive CTE for Hierarchical Data

Recursive CTEs are used to query hierarchical data, such as organizational charts or bill of materials.

Scenario: Traverse an organizational hierarchy to find all subordinates of a given manager.WITH EmployeeHierarchy AS (

```
  -- Anchor member: Starts with the initial manager
  SELECT
```

```
    employee_id,
    employee_name,
    manager_id,
    1 AS level
  FROM employees
  WHERE employee_id = 101 -- Start with a specific manager
  UNION ALL
  -- Recursive member: Finds direct reports and continues down the hierarchy
  SELECT
    e.employee_id,
    e.employee_name,
    e.manager_id,
    eh.level + 1
  FROM employees AS e
  INNER JOIN EmployeeHierarchy AS eh
    ON e.manager_id = eh.employee_id
)
SELECT
  employee_id,
  employee_name,
  manager_id,
```

```
    level

FROM EmployeeHierarchy;
```

Example 5: Multiple CTEs in a Single Query

You can define multiple CTEs within a single `WITH` clause, and subsequent CTEs can refer to previously defined CTEs.

Scenario: Find the average order amount for customers who placed more than 5 orders.WITH HighVolumeCustomers AS (

```
    SELECT

        customer_id,

        COUNT(order_id) AS total_orders

    FROM orders

    GROUP BY customer_id

    HAVING COUNT(order_id) > 5

),

CustomerOrderDetails AS (

    SELECT

        o.customer_id,

        o.order_amount

    FROM orders AS o

    INNER JOIN HighVolumeCustomers AS hvc

        ON o.customer_id = hvc.customer_id

)
```

```
SELECT

    cod.customer_id,

    AVG(cod.order_amount) AS average_order_amount

FROM CustomerOrderDetails AS cod

GROUP BY cod.customer_id;
```

These examples demonstrate the versatility and power of Common Table Expressions in writing more efficient and understandable SQL queries.